EventB_PO_Dataset

ชุดข้อมูล Event-B model พร้อมผลการพิสูจน์ Proof Obligation (PO) จาก Rodin
สำหรับงานทำนายจำนวน PO ที่ discharge ได้ด้วย LLM

48	28 : 19	1,035	35
model (generated 47 + model_001)	ผ่าน : fail (เฉพาะ 47 ตัวที่ generate)	PO ทั้งหมด (47 ตัว)	PO ที่ไม่ discharge

1. ภาพรวม

เป้าหมาย: ให้ LLM อ่าน Event-B model ทั้งตัว (`model.txt`) แล้วทำนายว่ามี PO ทั้งหมดกี่ข้อ และ discharge ได้กี่ข้อ label ทุกค่ามาจากการรัน Rodin จริง ไม่ได้คำนวณหรือเดาเอง

ที่มาของ model: sequence diagram ใน draw.io ถูกแปลงเป็น Event-B ด้วยเครื่องมือ **ChronoSeq-B** (แต่ละ message กลายเป็นคู่ event send/receive, fragment opt/alt/loop/par กลายเป็น guard และตัวนับ, ข้อกำหนดด้านเวลากลายเป็นตัวแปร `t_time / t_delay / t_success`)

การตั้งค่า prover ของ label หลัก: Rodin 3.9 บน Windows, auto-tactic ร่วมกับ **Atelier B provers (ML, PP)**

ประเภท (variant)	จำนวน	ผ่าน	fail	PO	PO fail
plain — ได้จาก generator โดยตรง	24	24	0	469	0
augmented — เพิ่ม invariant ที่พิสูจน์ได้	4	4	0	135	0
mutated — แก้ Event-B ให้ผิด	17	0	17	394	32
diagram_error — diagram ผิดเอง	2	0	2	37	3

model_001 (Payment) ไม่ได้อยู่ในตารางนี้ ดูหัวข้อ 7 ข้อจำกัด

2. โครงสร้างโฟลเดอร์

```
EventB_PO_Dataset/
├── README.pdf                  เอกสารนี้
├── dataset.jsonl               ไฟล์หลักสำหรับ train: 1 บรรทัด = 1 model
├── summary.csv                 ตารางสรุปทุก model สำหรับวิเคราะห์
│
├── models/
│   └── model_001 … model_048/
│       ├── model.txt           Event-B model เต็ม (input ของ LLM)
│       ├── label.json          ผล Rodin: total / discharged / failed
│       ├── po_details.json     ผลราย PO
│       ├── features.json       ตัวชี้วัดเชิงโครงสร้าง
│       └── meta.json           ที่มาของ model (ห้ามใช้เป็น input)
│
├── sources/
│   └── model_002 … model_048/
│       ├── <Case>.drawio.xml   sequence diagram ต้นทาง
│       └── <Case>_Rodin.zip     ไฟล์ที่ import เข้า Rodin จริง
│
├── rodin_raw/                  ไฟล์ดิบจาก Rodin (แหล่งที่มาของ label)
│   ├── ml_pp/                  Rodin 3.9 + Atelier B ML/PP  ← label หลัก
│   ├── basic_prover/           Rodin 3.9 ที่มีแค่ตัวพิสูจน์พื้นฐาน
│   └── original_payment/       ผล Rodin ของ model_001
│
├── labels_basic_prover/        label ชุดเปรียบเทียบ (basic prover)
│   ├── labels.jsonl
│   └── summary.csv
│
└── tools/
    ├── rebuild_labels.py       สร้าง label / jsonl / csv ใหม่จาก rodin_raw
    └── build_dataset_original.py   script ที่ใช้สร้าง dataset ครั้งแรก (อ้างอิง)
```

> **ID เรียงตามชื่อ case** ไม่ได้แยกกลุ่มผ่านหรือ fail จึงเดาผลจากเลข ID ไม่ได้ และ model ใน `sources/` กับ `rodin_raw/` จับคู่กับ `models/` ด้วย `meta.json` (ช่อง `source_dir` และ `rodin_raw_dir`)

3. ไฟล์ในแต่ละ model

3.1 model.txt — input ของ LLM

Event-B model เต็ม ไม่ตัด variable / invariant / event ออก ประกอบด้วย CONTEXT (sets, constants, axioms) และ MACHINE (variables, invariants, events พร้อม guard และ action) ตัวอย่างจาก `model_040` (SmartParking):

```
CONTEXT SmartParkingContext
SETS
    Objects; Messages; DataMessages
CONSTANTS
    Driver, ParkingGate, ParkingServer
    requestEntry_1, checkSlot_2, slotStatus_3, openGate_4, issueTicket_5, logEntry_6
    plateNo, slotInfo, ticketID
AXIOMS
    axm1: partition(Objects, {Driver}, {ParkingGate}, {ParkingServer})
    axm2: partition(Messages, {requestEntry_1}, {checkSlot_2}, {slotStatus_3}, {openGate_4},
{issueTicket_5}, {logEntry_6})
```

```
    axm3: partition(DataMessages, {plateNo}, {slotInfo}, {ticketID})
END
```

```
    sendrequestEntry_1
    WHEN
        grd1: requestEntry_1 ∉ sentMessages
        grd2: currentMessage = ∅
    THEN
        act1: sentMessages := sentMessages ∪ {requestEntry_1}
        act2: sender := sender ∪ {requestEntry_1 ↦ Driver}
        act3: receiver := receiver ∪ {requestEntry_1 ↦ ParkingGate}
        act4: receivedMessages := ∅
        act5: senderdataMessages := senderdataMessages ∪ {requestEntry_1 ↦ plateNo}
        act6: currentMessage := {requestEntry_1}
    END
```

3.2 label.json — ผล Rodin (target)

```
{
  "model_id": "model_002",
  "case": "AirportBaggage",
  "prover_config": "ml_pp",
  "total_po": 22,
  "discharged": 21,
  "failed": 1,
  "discharge_rate": 0.9545
}
```

field	ความหมาย
`total_po`	จำนวน PO ทั้งหมดที่ Rodin สร้าง (รวม machine และ context)
`discharged`	จำนวน PO ที่พิสูจน์ได้ (confidence = 1000)
`failed`	จำนวน PO ที่พิสูจน์ไม่ได้ = total_po − discharged
`discharge_rate`	discharged / total_po
`prover_config`	ชุดผลที่ใช้สร้าง label: `ml_pp` (หลัก) หรือ `basic_prover`

3.3 po_details.json — ผลราย PO

รายการของ PO ทุกข้อ ตัวอย่างข้อที่ไม่ผ่านใน `model_002` :

```
{
  "po": "INITIALISATION/inv19/INV",
  "component": "AirportBaggageInteractionMachine_1",
  "type": "INV",
  "confidence": 0,
  "discharged": false,
  "manual": false
}
```

field	ความหมาย
`po`	ชื่อ PO ในรูป `event/ส่วนที่พิสูจน์/ชนิด` เช่น `receiveX_3/inv5/INV`

field	ความหมาย
`type`	ชนิดของ PO: `INV` = event รักษา invariant, `FIS` = action เป็นไปได้ (feasibility)
`confidence`	ค่าจาก Rodin: 1000 = discharged, 0 = ยังพิสูจน์ไม่ได้
`discharged`	true เมื่อ confidence = 1000
`component`	ไฟล์ที่ PO อยู่ (machine หรือ context)
`manual`	true ถ้าพิสูจน์ด้วยมือ (ในชุดนี้เป็น false ทุกข้อ)

3.4 features.json — ตัวชี้วัดเชิงโครงสร้าง

ใช้วิเคราะห์ว่า feature ใดสัมพันธ์กับการ discharge หรือใช้เป็น input เสริมได้ ค่าทั้งหมดคำนวณจาก model เอง

feature	ความหมาย	คำนวณจาก
`n_objects`	จำนวน object (lifeline) ที่รับหรือส่ง message	diagram
`n_messages`	จำนวน message ใน diagram (แต่ละตัวกลายเป็น send + receive)	diagram
`n_data`	จำนวนชนิดของ data ที่ส่งไปกับ message	diagram
`fragments`	จำนวน fragment แยกตามชนิด เช่น `{"alt": 1, "loop": 1}`	diagram
`n_fragments`	จำนวน fragment ทั้งหมด	diagram
`max_nesting_depth`	ความลึกสูงสุดของการซ้อน fragment (0 = ไม่มี fragment)	diagram
`n_duration_constraints`	จำนวน message ที่มีช่วงเวลา `{a..b}`	diagram
`n_delay_constraints`	จำนวน message ที่มี delay `{t..t+n}`	diagram
`n_variables`	จำนวนตัวแปรใน MACHINE	Event-B
`n_invariants`	จำนวน invariant	Event-B
`n_axioms`	จำนวน axiom ใน CONTEXT	Event-B
`n_enum_constants`	จำนวนค่าคงที่แบบ enum (เช่น `VALID = 0`)	Event-B
`n_operations`	จำนวน event ทั้งหมด รวม INITIALISATION	Event-B
`n_send_receive_events`	จำนวน event send และ receive	Event-B
`n_checkloop_events`	จำนวน event checkloop (ตัวนับรอบของ loop)	Event-B
`n_guards`	จำนวน guard รวมทุก event	Event-B
`n_actions`	จำนวน action รวมทุก event รวม INITIALISATION	Event-B

3.5 meta.json — ที่มาของ model

> **ห้ามใช้ meta.json เป็น input ของ LLM** ไฟล์นี้บอกว่าตั้งใจให้ fail และแก้ตรงไหน ซึ่งเท่ากับเฉลย ใช้สำหรับวิเคราะห์ผลและตรวจสอบย้อนกลับเท่านั้น

```
{
  "model_id": "model_042",
  "case": "TaxiDispatch",
  "design_intent": "expected_fail",
  "variant": "mutated",
  "category": "model_error",
  "mutation": "send of findDriver_2 sets receivedMessages to {tripComplete_6} (not yet sent) instead
of ∅",
  "expected_failed_pos": [
    "sendfindDriver_2/inv5/INV"
  ],
  "expected_failed_count": 1,
  "source_dir": "sources/model_042",
  "rodin_raw_dir": "rodin_raw/ml_pp/TaxiDispatch_Project"
}
```

field	ความหมาย
`design_intent`	สิ่งที่ตั้งใจตอนสร้าง: `expected_pass` / `expected_fail` / `real_proof`
`variant`	plain / augmented / mutated / diagram_error / original (ดูหัวข้อ 1)
`category`	เฉพาะตัวที่ fail: `model_error` = model ผิดจริง, `non_inductive` = เป็นจริงในทุก state ที่ไปถึงได้แต่พิสูจน์ไม่ได้
`mutation`, `operations`	คำอธิบายและรายละเอียดการแก้ไข model
`expected_failed_pos`	PO ที่คาดว่าจะ fail ก่อนรัน Rodin (ใช้ตรวจความถูกต้องของ label)
`added_invariants`	เฉพาะ augmented: invariant ที่เพิ่มเข้าไป
`source_dir`, `rodin_raw_dir`	path ไปยังไฟล์ต้นทางและไฟล์ดิบของ Rodin

4. dataset.jsonl และ summary.csv

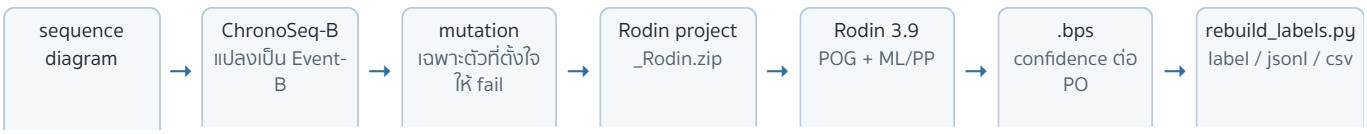
dataset.jsonl คือไฟล์หลักสำหรับ train มี 48 บรรทัด แต่ละบรรทัดมี `id, case, model` (ข้อความเต็มของ model.txt), feature ทุกตัวในหัวข้อ 3.4 และ `total_po, discharged, failed, discharge_rate` **ไม่มีข้อมูลจาก meta.json**

summary.csv ใช้วิเคราะห์ มีทุกคอลัมน์ข้างต้น (ยกเว้นข้อความ model) และเพิ่ม `design_intent, variant, expected_failed`

```
import json
rows = [json.loads(l) for l in open("dataset.jsonl", encoding="utf-8")]
data = [r for r in rows if r["id"] != "model_001"]     # ดูหัวข้อ 7

for r in data:
    x = r["model"]                        # input
    y = r["discharged"], r["total_po"]    # target
```

5. ขั้นตอนการสร้างข้อมูล



draw.io (.drawio.xml)						

1. **ออกแบบ diagram** ให้หลากหลาย: object 2–4 ตัว, message 4–11 ข้อความ, fragment ทุกชนิด ทั้งแบบไม่ซ้อนและซ้อนกัน สูงสุด 3 ชั้น และมีบางตัวที่ไม่มี fragment
2. **แปลงเป็น Event-B** ด้วย ChronoSeq-B ได้ทั้งข้อความ (`model.txt`) และ Rodin project
3. **ทำ mutation** กับตัวที่ตั้งใจให้ fail โดยแก้ทั้ง `model.txt` และไฟล์ `.bum` พร้อมกัน ให้สิ่งที่ LLM อ่านตรงกับสิ่งที่ Rodin พิสูจน์ (ยกเว้น diagram_error ที่ diagram ผิดเอง)
4. **รัน Rodin**: import ทั้ง 47 project แล้วให้ auto-prover (ML, PP) ทำงาน
5. **อ่านผล** จากไฟล์ `.bps` : PO ที่ `confidence = 1000` นับเป็น discharged
6. **ตรวจสอบ**: เทียบ PO ที่ fail กับ `expected_failed_pos` ผลตรงกันทุก model (mismatch = 0)

สร้าง label ใหม่จากไฟล์ดิบ

```
cd EventB_PO_Dataset
python tools/rebuild_labels.py                                        # label หลัก (ml_pp)
python tools/rebuild_labels.py --config basic_prover --out labels_basic_prover
```

6. สถิติ

จำนวน object

object	model
2	13
3	16
4	18

ช่วงของ feature (47 ตัว)

feature	ต่ำสุด–สูงสุด
`n_messages`	4–11
`n_variables`	13–24
`n_invariants`	13–24
`n_operations`	9–23
`n_guards`	31–95
`total_po`	11–54

จำนวน PO ที่ fail ต่อ model

PO ที่ fail	model
1	13
2	3
3	1
6	1
7	1

ชนิดของ PO

ชนิด	ทั้งหมด	fail
INV	885	32
FIS	150	3

ตำแหน่งที่ PO fail

event	PO fail
receive	17
send	9
INITIALISATION	7
checkloop	2

model ที่ตั้งใจให้ fail (19 ตัว)

id	case	ประเภท	แก้อะไร	PO fail
model_002	AirportBaggage	model ผิดจริง	เพิ่ม invariant เวลา 3 ข้อ แต่ตั้งค่าเริ่มต้น `t_delay_2 := −2`	1
model_004	BankTransfer	ไม่ inductive	ลบ guard `m ∈ sentMessages` ออกจาก receive 2 event	2
model_005	BatteryChargeCycle	model ผิดจริง	เพิ่ม `pulseNo ≥ 0`, `pulseNo ≤ 8` แต่ checkloop ลดค่า counter แทนการเพิ่ม	1
model_008	ChatMessaging	ไม่ inductive	เพิ่ม `receiverdataMessages ⊆ senderdataMessages` (เป็นจริงแต่ไม่ inductive)	6
model_011	ColdChainMonitor	ไม่ inductive	เพิ่ม `t_time_2 ≥ 0` (ผ่าน) และ `t_success_2 ≥ 0` ซึ่งขาด `t_delay_2 ≥ 0` ช่วยพิสูจน์	1
model_012	ConferenceRoomBooking	model ผิดจริง	เพิ่ม `dom(receiverdataMessages) ⊆ receivedMessages` แต่ทุก send ล้าง `receivedMessages` เป็น ∅	7
model_013	CourseEnrollment	model ผิดจริง	เพิ่ม `currentMessage ⊆ sentMessages` (ผ่าน) และ `receivedMessages ⊆ dom(receiverdataMessages)` (ผิดกับ message ที่ไม่มี data)	3
model_022	GymCheckIn	model ผิดจริง	ค่าเริ่มต้น `receivedMessages := {scanQR_1}` ทั้งที่ยังไม่มีการส่ง	1
model_026	LabSampleTesting	model ผิดจริง	ช่วงเวลาใน diagram เขียนกลับหัว 2 จุด `{12..4}` และ `{9..6}`	2
model_028	LoginRateLimiter	model ผิดจริง	เพิ่ม `failCount ≤ 5` แต่ checkloop ใช้ guard `≤ 5` แทน `< 5` (off-by-one)	1
model_029	MedicationDispenser	model ผิดจริง	เพิ่ม `currentMessage ⊆ sentMessages` แต่ send ของ `logDose_4` ตั้ง `currentMessage := {logAck_5}`	1
model_030	OnlineAuction	ไม่ inductive	ลบ guard `m ∈ sentMessages` ออกจาก receive 1 event	1
model_033	ParcelLocker	model ผิดจริง	ช่วงเวลาใน diagram เขียนกลับหัว `{10..3}` ทำให้ไม่มีค่าเริ่มต้นที่เป็นไปได้	1
model_034	PrinterQueue	model ผิดจริง	เพิ่ม `jobCount ≤ 5` (ผ่าน) และ `retryPrint ≤ 3` แต่ตั้งค่าเริ่มต้น `retryPrint := 4`	1
model_039	SmartMeterBilling	model ผิดจริง	receive ของ `generateBill_3` บันทึก `payBill_4` ที่ยังไม่ถูกส่ง	1
model_041	SmartPlugControl	model ผิดจริง	receive ของ `turnOff_3` บันทึก `turnOn_5` ของอีก branch	1
model_042	TaxiDispatch	model ผิดจริง	send ของ `findDriver_2` ตั้ง `receivedMessages := {tripComplete_6}` แทน `∅`	1
model_044	ToolRentalLocker	model ผิดจริง	ค่าเริ่มต้น `receivedMessages` ผิด และ receive ของ `verifyCode_4` บันทึก message ผิดตัว	2
model_045	TrafficLightController	model ผิดจริง	เพิ่ม `t_success_1 ≤ 20` แต่ค่าจริงขึ้นได้ถึง 30+5	1

model ผ่านที่เพิ่ม invariant (augmented)

เพิ่มเพื่อไม่ให้ "มี invariant เพิ่ม" กลายเป็นสัญญาณว่า fail และจับคู่เทียบกับตัวที่ fail เช่น FitnessTracker มี `t_delay ≥ 0` ช่วยพิสูจน์จึงผ่าน ส่วน ColdChainMonitor ขาดข้อนี้จึง fail

id	case	invariant ที่เพิ่ม	PO fail

model_006	BloodBankRequest	`currentMessage ⊆ sentMessages`
		`t_time_4 ≥ 0`
		`t_delay_4 ≥ 0`
			0

model_020	FitnessTracker	`syncRound ≤ 10`
		`t_time_3 ≥ 0`
		`t_delay_3 ≥ 0`
		`t_success_3 ≥ 0`
			0

model_024	HotelCheckIn	`currentMessage ⊆ sentMessages` `receivedMessages ⊆ dom(receiver)`	0
model_036	ServerAutoScaling	`checkNo ≤ 12` `currentMessage ⊆ sentMessages`	0

7. ข้อจำกัดและข้อควรระวัง

> **model_001 (Payment) ใช้ทดสอบ pipeline เท่านั้น** ไฟล์ `.bum` ต้นฉบับหายไป `model.txt` จึงประกอบกลับจากไฟล์ของ Rodin ส่วน EVENTS มีแค่ชื่อ event ไม่มี guard/action, feature มีไม่ครบ และชื่อ set ไม่ตรงกับตัวอื่น label (24/24) ถูกต้อง แต่ไม่ควรใช้วัดความแม่นยำของ LLM

> **label ขึ้นกับ prover ที่ใช้** model ชุดเดียวกัน ถ้าใช้แค่ตัวพิสูจน์พื้นฐานของ Rodin จะได้ผ่าน 24 : fail 23 และ PO fail 71 ข้อ เทียบกับ ML/PP ที่ได้ 28 : 19 และ 35 ข้อ ต้องระบุ prover ทุกครั้งที่รายงานผล ชุด basic อยู่ใน `labels_basic_prover/`

id	case	variant	PO fail (basic)	PO fail (ML/PP)
model_002	AirportBaggage	mutated	3	1
model_005	BatteryChargeCycle	mutated	2	1
model_006	BloodBankRequest	augmented	1	0
model_008	ChatMessaging	mutated	12	6
model_011	ColdChainMonitor	mutated	2	1
model_012	ConferenceRoomBooking	mutated	14	7
model_013	CourseEnrollment	mutated	8	3
model_020	FitnessTracker	augmented	3	0
model_024	HotelCheckIn	augmented	7	0
model_034	PrinterQueue	mutated	3	1
model_036	ServerAutoScaling	augmented	1	0

- **เป็นข้อมูลสังเคราะห์** model มาจาก diagram ที่ออกแบบขึ้นและ mutation ที่ตั้งใจใส่ อาจไม่ครอบคลุมความผิดพลาดทุกแบบในงานจริง
- **ขนาดเล็ก** 47 model เหมาะกับ few-shot, RAG หรือการประเมิน มากกว่าการ fine-tune
- **ไฟล์ที่ห้ามใช้เป็น input:** `meta.json`, `sources/`, `rodin_raw/` เพราะมีข้อมูล mutation หรือผลการพิสูจน์
- **ชื่อ case ไม่บอกผล** แต่ชื่อ event และตัวแปรยังเป็นภาษาธรรมชาติ ถ้าต้องการตัดผลของชื่อออก ให้เปลี่ยนชื่อก่อนส่งเข้า LLM
- **script ที่ใช้สร้าง diagram และ mutation ไม่ได้รวมอยู่ในแพ็กเกจ** แต่ข้อมูลที่ใช้ทำซ้ำอยู่ครบ ได้แก่ diagram ใน `sources/` รายละเอียดการแก้ใน `meta.json` (`operations`) และ Rodin project ที่ import จริง
- **พฤติกรรมของ generator:** alt ที่ใช้เงื่อนไขแบบเท่ากับ (เช่น `== CRITICAL`) จะได้ค่าเริ่มต้นคงที่ ทำให้ model checking เข้าได้แค่ branch เดียว ไม่มีผลกับจำนวน PO